MASTER NLP TRONG 30 NGÀY

Roadmap thực chiến cho Olympic Trí tuệ nhân tạo Sinh viên (OlpAI)

Stack: scikit-learn -> PyTorch -> Hugging Face Transformers

Mục tiêu: Sau 30 ngày, bạn có thể nhận một dataset NLP lạ, xác định đúng task/metric/validation, dựng baseline nhanh, fine-tune Transformer, xử lý tiếng Việt, NER và sentence-pair, tối ưu GPU, phân tích lỗi và tạo submission reproducible trong thời gian contest.

Bản chi tiết - thiết kế để học 3-4 giờ/ngày

Cập nhật: 29/09/2026

Cách sử dụng tài liệu này

Bối cảnh OlpAI 2026: Quy chế hiện công bố đề thi có các mảng ML, CV và NLP; đội gồm 2-3 sinh viên. Roadmap này tối ưu cho vai trò thành viên phụ trách NLP, nhưng không giả định bài NLP chỉ là sentiment classification.

• Quy chế/Kế hoạch OlpAI 2026 - Nguồn chính thức để kiểm tra thay đổi về thể lệ và môi trường thi.

Nhịp học mỗi ngày

Khối A - Lý thuyết	45-60 phút: đọc đúng mục được chỉ định, tự viết lại công thức/khái niệm bằng lời của mình.
Khối B - Guided coding	60-90 phút: triển khai pipeline theo checklist, chỉ xem code tham khảo sau khi đã tự thử.
Khối C - Experiment	60-90 phút: chạy 2-4 thí nghiệm có giả thuyết, ghi metric + thời gian + kết luận.
Khối D - Review	15 phút: lưu notebook, README ngắn, experiment.csv và 3 lỗi/bài học lớn nhất trong ngày.

Quy tắc bắt buộc khi luyện contest

• Không tối ưu model trước khi có baseline hợp lệ.

• Không nhìn test label/leaderboard để chọn hyperparameter. Mọi quyết định phải dựa trên validation.

• Mỗi experiment chỉ thay đổi 1-2 yếu tố lớn để biết vì sao metric tăng/giảm.

• Ghi seed, split, metric, thời gian train, model, preprocessing và file output.

• Từ ngày 15 trở đi luôn đo cả chất lượng và chi phí: thời gian, VRAM, số epoch, kích thước model.

• Với tiếng Việt: không mặc định “clean càng nhiều càng tốt”; dấu, viết tắt, emoji, URL có thể mang tín hiệu.

Cấu trúc thư mục luyện tập

```
nlp_olpai/
  day01_eda_imdb/
    notebook.ipynb
    README.md
    experiments.csv
    outputs/
  day02_vietnamese_preprocess/
  ...
  templates/
    classical_text_cls.py
    hf_sequence_cls.py
    hf_token_cls.py
    submission_check.py
```

Mẫu experiment log

exp_id	E01, E02, ...
split/seed	stratified 80/20, seed=42
model	LinearSVC / DistilBERT / PhoBERT ...

changes	n-gram, LR, max_length, class weights ...
metric	Macro-F1/Accuracy/AUC/Seqeval-F1 tùy bài
runtime	phút + GPU/VRAM nếu có
kết luận	Tăng/giảm bao nhiêu? Vì sao? Thử gì tiếp?

Bản đồ 30 ngày

Giai đoạn	Trọng tâm	Bạn phải làm được
Tuần 1	Classical NLP & validation	EDA -> cleaning -> TF-IDF -> n-gram -> linear models -> metrics -> mini contest
Tuần 2	Deep NLP bằng PyTorch	Dataset/DataLoader -> embeddings -> TextCNN -> RNN/LSTM -> attention -> mini Transformer
Tuần 3	Pretrained Transformers	BERT/tokenizer -> Hugging Face -> fine-tune -> PhoBERT/XLM-R -> tuning -> multi-label -> contest
Tuần 4	Task breadth & contest engineering	NER -> sentence pair -> semantic similarity -> noise/augmentation -> CV/ensemble -> GPU -> 3 mock contests

NGÀY 01 - EDA văn bản và tư duy nhận dạng bài toán

Dataset	IMDb Large Movie Review Dataset
Cách lấy dữ liệu	Hugging Face: load_dataset("stanfordnlp/imdb")
Metric chính	Không chấm metric model; chấm chất lượng EDA và split.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Hiểu cấu trúc một bài text classification; trong 30 phút đầu phải mô tả được dữ liệu, nhãn, độ dài, duplicate và rủi ro split.

1. Học lý thuyết gì?

• Text classification là gì; sample, feature, label; train/validation/test.

• EDA cho text: class balance, null/duplicate, ký tự, word/token length, outlier length.

• Data leakage cơ bản: không fit vectorizer/preprocessing thống kê trên toàn bộ dữ liệu trước khi split.

• Khác biệt giữa Accuracy và F1 ở mức khái niệm.

2. Triển khai từng bước

1. Load train/test, lấy 10-20k train để chạy nhanh.

2. Tạo DataFrame: text, label; kiểm tra null/duplicate.

3. Tính char_len, word_len; mô tả median/p90/p95/p99 theo class.

4. In 10 câu ngắn nhất/dài nhất; đọc tay ít nhất 20 mẫu.

5. Stratified train/validation 80/20, seed=42.

6. Lưu report_eda.md: 5 quan sát + 3 giả thuyết model/preprocessing.

3. Experiments bắt buộc

• E01: split random vs stratified - so phân bố nhãn.

• E02: giữ duplicate vs drop exact duplicate - ghi số mẫu thay đổi.

• E03: so độ dài hai class để xem length có thể là tín hiệu.

4. Tiêu chí PASS

PASS: Bạn giải thích được dataset trong <=5 phút và tạo split reproducible; report có ít nhất 5 insight cụ thể, không chỉ "data khá cân bằng".

5. Nguồn học lý thuyết

• Stanford CS224N - Đọc phần introduction/history để có bức tranh NLP.

• Hugging Face Datasets - Loading - Cách load dataset và inspect split.

6. Nguồn tham khảo code / API

• IMDb dataset card - Xem fields/splits và cách load.

• scikit-learn text tutorial - Xem workflow text classification cổ điển.

7. Lỗi hay mắc

• Đếm độ dài sau khi preprocessing mà chưa biết preprocessing có hợp lý không.

• Đọc toàn bộ test/validation rồi vô tình điều chỉnh rule theo test.

• EDA chỉ vẽ biểu đồ mà không rút ra quyết định.

8. Deliverables cuối ngày

• day01/notebook.ipynb

• report_eda.md

• split_indices.csv hoặc seed/config rõ ràng

9. Stretch goal (chỉ làm sau khi PASS)

• Tạo hàm profile_text_dataset(df, text_col, label_col) tái sử dụng cho mọi ngày sau.

NGÀY 02 - Tiền xử lý tiếng Việt: Unicode, dấu, URL, emoji, tokenization

Dataset	UIT-VSFC - Vietnamese Students' Feedback Corpus
Cách lấy dữ liệu	GitHub dataset; tải các file train/dev/test. Nếu dùng PhoBERT về sau, giữ bản raw song song bản đã word-segment.
Metric chính	Chưa cần model; mục tiêu là preprocessing an toàn và modular.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết cái gì nên giữ/xóa trong text tiếng Việt và xây preprocessing dưới dạng các bước bật/tắt được để A/B test.

1. Học lý thuyết gì?

• Unicode normalization NFC/NFKC và lỗi ký tự tiếng Việt.

• lowercase, punctuation, number, URL, email, emoji, repeated chars, teencode/viết tắt.

• Tokenization khác word segmentation; tiếng Việt có từ nhiều âm tiết.

• Nguyên tắc contest: preprocessing phải là hypothesis, không phải nghi thức bắt buộc.

2. Triển khai từng bước

1. Đọc 100 feedback ngẫu nhiên; ghi các pattern tiếng Việt đặc thù.

2. Viết normalize_unicode(), normalize_space(), replace_url(), replace_email(), normalize_repeated_chars().

3. Tạo 3 pipeline: RAW; LIGHT (unicode+space+URL token); AGGRESSIVE (thêm lowercase/punctuation/stopword).

4. Tạo bảng before/after cho 30 mẫu.

5. Lưu text_raw và text_clean để ngày 6 so metric thật.

3. Experiments bắt buộc

• E01 RAW vs LIGHT: thống kê vocabulary size/avg length.

• E02 LIGHT vs AGGRESSIVE: bao nhiêu token bị mất? đọc tay 20 câu.

• E03 word segmentation bằng VnCoreNLP trên 500 mẫu, so số token và ví dụ.

4. Tiêu chí PASS

PASS: Pipeline không phá dấu tiếng Việt, xử lý được URL/emoji/space; bạn giải thích được vì sao chưa nên remove stopword mặc định.

5. Nguồn học lý thuyết

• PhoBERT official repo - Đọc Notes về VnCoreNLP word segmentation.

• CS224N - Ôn tokenization/representation ở mức khái niệm.

6. Nguồn tham khảo code / API

• UIT-VSFC repository - Dataset tiếng Việt và cấu trúc nhãn.

• PhoBERT usage - Code word segmentation và tokenizer/model.

7. Lỗi hay mắc

• Xóa emoji/!/? dù sentiment có thể nằm ở đó.

• Bỏ dấu tiếng Việt.

• Word-segment rồi lại tách bằng space theo rule khác, làm sai token.

8. Deliverables cuối ngày

• preprocess.py

• preprocess_comparison.csv

• 30 ví dụ before/after có nhận xét

9. Stretch goal (chỉ làm sau khi PASS)

• Thêm unit tests cho 10 chuỗi "khó": URL, emoji, nhiều dấu cách, dấu câu lặp, chữ hoa, Unicode tổ hợp.

NGÀY 03 - Bag-of-Words, TF, IDF và TF-IDF

Dataset	SMS Spam Collection (UCI)
Cách lấy dữ liệu	UCI: 5,574 SMS; có thể dùng ucimlrepo hoặc tải file SMSSpamCollection.
Metric chính	Macro-F1 + spam-class precision/recall; accuracy chỉ là metric phụ.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Tự giải thích và kiểm chứng TF-IDF; từ raw text tạo sparse matrix đúng cách mà không leakage.

1. Học lý thuyết gì?

• BoW/CountVectorizer; vocabulary và sparse matrix.

• TF, document frequency, IDF, TF-IDF; L2 normalization.

• fit vs transform: vectorizer chỉ fit trên train.

• min_df, max_df, max_features, sublinear_tf.

2. Triển khai từng bước

1. Tự tính TF-IDF bằng tay cho 3-5 câu rất nhỏ.

2. Dùng CountVectorizer, xem 20 features và shape.

3. Dùng TfidfVectorizer, xem idf_ của từ phổ biến/hiếm.

4. Split stratified; fit_transform(train), transform(valid).

5. Train LogisticRegression baseline chỉ để kiểm chứng representation.

3. Experiments bắt buộc

• E01 Count vs TF-IDF.

• E02 smooth_idf on/off hoặc sublinear_tf True/False.

• E03 min_df=1/2/3; ghi số features + F1.

4. Tiêu chí PASS

PASS: Bạn giải thích được vì sao fit TF-IDF trên toàn data là leakage; notebook có sparse matrix, feature inspection và baseline hợp lệ.

5. Nguồn học lý thuyết

• scikit-learn Working With Text Data - BoW -> TF-IDF -> classifier.

• TfidfVectorizer docs - Tham số chính và hành vi analyzer/ngram/min_df.

6. Nguồn tham khảo code / API

• SMS Spam UCI - Dataset + hướng dẫn tải.

• TfidfVectorizer examples - Code API chuẩn.

7. Lỗi hay mắc

• Dùng .toarray() cho matrix lớn gây tốn RAM.

• fit_transform validation/test.

• Chọn max_features quá nhỏ mà không quan sát feature loss.

8. Deliverables cuối ngày

• tfidf_from_scratch.ipynb

• tfidf_sklearn.ipynb

• experiments.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Viết hàm top_tfidf_terms(text, vectorizer, k=10) để debug một prediction.

NGÀY 04 - Word n-gram, character n-gram và Pipeline

Dataset	SMS Spam Collection
Cách lấy dữ liệu	Tái sử dụng split ngày 3 để so công bằng.
Metric chính	Macro-F1; ghi thêm train time + số features.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết khi nào char n-gram thắng word n-gram; đóng gói vectorizer + classifier thành Pipeline chống leakage.

1. Học lý thuyết gì?

• Unigram/bigram/trigram và trade-off sparsity.

• character n-gram/char_wb: hữu ích với typo, biến thể chính tả, từ hiếm.

• Pipeline trong scikit-learn và cách tune tham số.

• Vì sao preprocessing mạnh đôi khi làm char n-gram yếu đi.

2. Triển khai từng bước

1. Tạo Pipeline(TfidfVectorizer, LinearSVC).

2. Chạy word (1,1), word (1,2), char_wb (3,5), char_wb (2,6).

3. Tạo hybrid bằng FeatureUnion word + char nếu còn thời gian.

4. Đo feature count, RAM tương đối, train time, Macro-F1.

5. Đọc 30 lỗi nơi char đúng nhưng word sai và ngược lại.

3. Experiments bắt buộc

• E01 word unigram.

• E02 word (1,2).

• E03 char_wb (3,5).

• E04 hybrid word+char.

4. Tiêu chí PASS

PASS: Có bảng so sánh ít nhất 4 representations và một kết luận dựa trên error analysis, không chỉ chọn điểm cao nhất.

5. Nguồn học lý thuyết

• TfidfVectorizer docs - Đọc analyzer, ngram_range, char_wb.

• scikit-learn text tutorial - Pipeline và parameter tuning.

6. Nguồn tham khảo code / API

• TfidfVectorizer API examples - Mẫu code vectorization.

• Working With Text Data - Mẫu pipeline classifier.

7. Lỗi hay mắc

• Grid search quá rộng trên ngày 4.

• So model trên split khác nhau.

• Dùng cleaning khác nhau nhưng không ghi lại.

8. Deliverables cuối ngày

• ngram_comparison.ipynb

• error_cases_word_vs_char.csv

• pipeline_classical.py

9. Stretch goal (chỉ làm sau khi PASS)

• Thử classifier chỉ dùng character n-gram trên chuỗi đã cố tình thêm typo để xem robustness.

NGÀY 05 - Naive Bayes, Logistic Regression và LinearSVC

Dataset	20 Newsgroups (scikit-learn) hoặc IMDb nếu muốn sentiment
Cách lấy dữ liệu	scikit-learn fetch_20newsgroups; chọn 4-6 categories để chạy nhanh.
Metric chính	Macro-F1 cho multiclass + per-class F1 + runtime.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết ưu/nhược của ba baseline kinh điển và chọn model theo representation/metric thay vì cảm tính.

1. Học lý thuyết gì?

• MultinomialNB: giả định bag-of-counts và smoothing alpha.

• Logistic Regression: linear decision boundary, regularization C.

• LinearSVC: margin, thường mạnh với high-dimensional sparse text.

• Probability vs decision_function; calibration chỉ khi thực sự cần probability.

2. Triển khai từng bước

1. Chuẩn hóa một split cố định.

2. TF-IDF word (1,2) cho cả 3 model.

3. Tune nhẹ alpha/C (3-4 giá trị).

4. Vẽ confusion matrix và xem top errors theo class.

5. Với Logistic: xem top positive/negative coefficients; với multiclass xem top features/class.

3. Experiments bắt buộc

• NB: alpha 0.1/0.5/1.0.

• LogReg: C 0.5/1/2.

• LinearSVC: C 0.5/1/2.

4. Tiêu chí PASS

PASS: Bạn chọn được baseline mặc định cho contest text sparse và giải thích bằng metric + runtime + lỗi.

5. Nguồn học lý thuyết

• scikit-learn text tutorial - Classifier tuyến tính trên text.

• CS224N - Ôn linear classifiers/neural basics nếu cần.

6. Nguồn tham khảo code / API

• Working With Text Data - Có example classifier + evaluation.

• LinearSVC docs - API và C/class_weight.

7. Lỗi hay mắc

• Tin rằng model phức tạp luôn thắng.

• Chỉ nhìn accuracy và bỏ class yếu.

• Tune C trên test.

8. Deliverables cuối ngày

• model_baselines.ipynb

• top_features.csv

• confusion_matrix.png

9. Stretch goal (chỉ làm sau khi PASS)

• Đóng gói function train_eval_classical(config) nhận vectorizer/model params từ dict.

NGÀY 06 - Metrics, imbalance, threshold và validation

Dataset	dair-ai/emotion
Cách lấy dữ liệu	Hugging Face: load_dataset("dair-ai/emotion", "split") hoặc cấu hình mặc định theo dataset card.
Metric chính	Macro-F1 là metric chính trong lab; accuracy/weighted-F1 để so.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Chọn đúng metric và hiểu tại sao accuracy có thể lừa; biết threshold tuning chỉ làm trên validation.

1. Học lý thuyết gì?

• Precision, Recall, F1; macro/micro/weighted.

• Confusion matrix; class support.

• Class imbalance và class_weight.

• Threshold tuning binary/multilabel; multiclass argmax.

• Stratified split và variance do seed.

2. Triển khai từng bước

1. EDA phân bố 6 emotion classes.

2. TF-IDF + LinearSVC/LogReg baseline.

3. Đo accuracy, macro-F1, weighted-F1.

4. Tạo bản imbalance nhân tạo (downsample một class) để quan sát accuracy vs macro-F1.

5. Chạy 3 seeds và ghi mean/std.

3. Experiments bắt buộc

• E01 no class_weight.

• E02 class_weight="balanced".

• E03 3 random seeds cùng config.

4. Tiêu chí PASS

PASS: Bạn giải thích được macro-F1 khác weighted-F1 và báo cáo mean/std thay vì một con số “đẹp”.

5. Nguồn học lý thuyết

• scikit-learn model evaluation - Metric definitions.

• Emotion dataset card - 6-class emotion classification.

6. Nguồn tham khảo code / API

• classification_report - Per-class precision/recall/F1.

• StratifiedKFold - Split preserving class distribution.

7. Lỗi hay mắc

• Tuning threshold trên test.

• Báo cáo weighted-F1 khi đề dùng macro-F1.

• Một seed duy nhất rồi kết luận chắc chắn.

8. Deliverables cuối ngày

• metrics_lab.ipynb

• seed_variance.csv

• metric_cheatsheet.md

9. Stretch goal (chỉ làm sau khi PASS)

• Viết hàm evaluate_classification(y_true, pred/proba) trả dict metric + per-class table.

NGÀY 07 - Mini Contest #1 - Classical NLP dưới áp lực thời gian

Dataset	AG News
Cách lấy dữ liệu	Hugging Face: load_dataset("fancyzhx/ag_news"); 4 class news topic.
Metric chính	Accuracy + Macro-F1; ưu tiên metric được đặt trước khi bắt đầu.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Trong 2 giờ: tự EDA -> split -> baseline -> 3 experiments -> final prediction, không xem notebook lời giải trước.

1. Học lý thuyết gì?

• Ôn nhanh toàn tuần; không học lý thuyết mới.

• Chiến thuật timebox: 15m read/EDA; 20m baseline; 60m experiments; 20m error analysis; 5m package.

2. Triển khai từng bước

1. 0:00-0:15: inspect data, metric, split.

2. 0:15-0:35: TF-IDF word + LinearSVC baseline.

3. 0:35-1:20: char n-gram/hybrid/C/preprocess.

4. 1:20-1:40: đọc 30 errors, chọn đúng 1 cải tiến.

5. 1:40-2:00: retrain config tốt nhất, save model/config/prediction.

3. Experiments bắt buộc

• Baseline word unigram.

• word (1,2).

• char_wb.

• hybrid hoặc tune C - chỉ nếu còn giờ.

4. Tiêu chí PASS

PASS: PASS nếu baseline <=20 phút sau khi split; >=3 experiment có log; final notebook chạy từ đầu không lỗi.

5. Nguồn học lý thuyết

• scikit-learn text tutorial - Chỉ được dùng như documentation, không copy một notebook hoàn chỉnh.

• AG News dataset - Dataset card.

6. Nguồn tham khảo code / API

• TfidfVectorizer docs - Tra API khi cần.

• LinearSVC docs - Tra API khi cần.

7. Lỗi hay mắc

• Tốn 30 phút cleaning trước baseline.

• GridSearchCV hàng chục combinations.

• Không lưu prediction/config cuối.

8. Deliverables cuối ngày

• contest_day07.ipynb

• experiments.csv

• final_config.json

• predictions.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Tự giới hạn còn 90 phút và lặp lại từ notebook trống vào hôm sau nếu baseline >20 phút.

NGÀY 08 - PyTorch nền tảng cho NLP: Dataset, DataLoader, training loop

Dataset	SMS Spam Collection
Cách lấy dữ liệu	Tái sử dụng dataset nhỏ để tập framework, tránh GPU/compute che khuất bug.
Metric chính	Validation F1 + train/epoch time; mục tiêu hiểu code hơn là thắng TF-IDF.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Tự viết training loop PyTorch và hiểu forward -> loss -> backward -> optimizer.step; không dùng Trainer.

1. Học lý thuyết gì?

• Tensor shape/dtype/device.

• torch.utils.data.Dataset và DataLoader.

• nn.Module, forward, CrossEntropyLoss/BCEWithLogitsLoss.

• optimizer.zero_grad -> loss.backward -> optimizer.step.

• train/eval/no_grad và reproducibility seed.

2. Triển khai từng bước

1. Tạo vocab đơn giản từ train only; encode token IDs.

2. Custom TextDataset trả input_ids, length, label.

3. collate_fn pad batch.

4. Model: Embedding -> mean pooling mask-aware -> Linear.

5. Viết train_one_epoch(), evaluate(); lưu best checkpoint theo validation F1.

6. So với TF-IDF baseline ngày 3.

3. Experiments bắt buộc

• E01 embedding_dim=64.

• E02 128.

• E03 dropout 0/0.2.

4. Tiêu chí PASS

PASS: Bạn có thể tự viết training loop từ notebook trắng, biết chuyển train/eval mode và không average padding tokens.

5. Nguồn học lý thuyết

• PyTorch Learn the Basics - Dataset/DataLoader/model/autograd/optimization.

• Deep Learning for NLP with PyTorch - NLP-specific PyTorch tutorial.

6. Nguồn tham khảo code / API

• Datasets & DataLoaders - Custom Dataset/DataLoader patterns.

• PyTorch NLP tutorials - Embedding + training examples.

7. Lỗi hay mắc

• Quên model.eval() khi validation.

• Không mask PAD khi mean pooling.

• Vocab build từ train+validation gây leakage nhẹ.

8. Deliverables cuối ngày

• pytorch_text_baseline.ipynb

• model.py

• train.py

• best.pt

9. Stretch goal (chỉ làm sau khi PASS)

• Thêm gradient norm logging và early stopping patience=2.

NGÀY 09 - Word Embeddings: Word2Vec/FastText và cosine similarity

Dataset	WikiText-2
Cách lấy dữ liệu	Hugging Face: load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")
Metric chính	Intrinsic qualitative + downstream F1 nhỏ; không cần đạt SOTA.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Hiểu static embedding, OOV/subword và tự kiểm tra không gian vector bằng nearest neighbors.

1. Học lý thuyết gì?

• Distributional hypothesis.

• CBOW vs Skip-gram; negative sampling ở mức trực giác.

• GloVe/FastText: FastText dùng subword n-grams nên xử lý OOV tốt hơn.

• Cosine similarity và hạn chế của static embeddings/polysemy.

2. Triển khai từng bước

1. Lấy corpus WikiText-2 subset; tokenize cơ bản.

2. Dùng gensim Word2Vec hoặc FastText chỉ cho lab representation (không thay framework DL chính).

3. Tìm nearest neighbors cho 20 từ.

4. Tạo 5 cặp synonym/related/unrelated và so cosine.

5. Thử misspelling/rare form với FastText.

6. Nạp pretrained vector vào nn.Embedding và freeze/unfreeze trên một classifier nhỏ.

3. Experiments bắt buộc

• E01 random embedding.

• E02 pretrained/fitted Word2Vec frozen.

• E03 unfreeze embedding.

4. Tiêu chí PASS

PASS: Bạn giải thích được Word2Vec khác contextual embedding; có ví dụ OOV/FastText và kết quả nearest-neighbor hợp lý.

5. Nguồn học lý thuyết

• CS224N Word Vectors - Slides/notes Word Vectors.

• PyTorch Word Embeddings tutorial - Embedding trong PyTorch.

6. Nguồn tham khảo code / API

• WikiText dataset - Corpus luyện embedding.

• PyTorch word embeddings code - Code reference.

7. Lỗi hay mắc

• Đánh giá embedding chỉ bằng vài từ “đẹp”.

• Nhầm Word2Vec với BERT embedding.

• Không giữ PAD index cố định khi import weights.

8. Deliverables cuối ngày

• embedding_lab.ipynb

• nearest_neighbors.csv

• notes_static_vs_contextual.md

9. Stretch goal (chỉ làm sau khi PASS)

• Thử PCA/t-SNE chỉ để trực quan hóa 100-300 từ; không dùng plot để “chứng minh” chất lượng.

NGÀY 10 - TextCNN cho phân loại văn bản

Dataset	AG News
Cách lấy dữ liệu	Dùng 20k-40k train samples để iteration nhanh; tokenizer/vocab từ tuần 2.
Metric chính	Accuracy/Macro-F1 + time/epoch.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Implement TextCNN và hiểu Conv1D bắt local n-gram patterns như thế nào.

1. Học lý thuyết gì?

• Embedding tensor shape [B,L,E].

• Conv1D và kernel size như n-gram detector.

• Global max pooling.

• Dropout và overfitting.

• Padding ảnh hưởng convolution.

2. Triển khai từng bước

1. Chuẩn bị batches fixed/max length hợp lý (p95 length).

2. Model: Embedding -> Conv1d kernels 3/4/5 -> ReLU -> global max pool -> concat -> dropout -> Linear.

3. Train 3-5 epochs, early stopping.

4. Log train/val loss + accuracy/F1.

5. So error và runtime với TF-IDF LinearSVC ngày 7.

3. Experiments bắt buộc

• E01 kernels [3,4,5], 100 filters.

• E02 dropout 0.2 vs 0.5.

• E03 max_len p90 vs p95.

4. Tiêu chí PASS

PASS: Model train ổn định, không lỗi shape; bạn chỉ ra được ít nhất 10 trường hợp TextCNN sửa lỗi của TF-IDF hoặc ngược lại.

5. Nguồn học lý thuyết

• PyTorch NLP tutorials - Ôn nn.Module/training.

• CS224N - Tham khảo CNN for sentence classification nếu có trong schedule/materials.

6. Nguồn tham khảo code / API

• PyTorch Conv1d docs - API/shape.

• AG News dataset - Dataset.

7. Lỗi hay mắc

• Transpose sai B/L/E khi Conv1d.

• Max_len quá lớn làm chậm mà không tăng metric.

• Không shuffle train DataLoader.

8. Deliverables cuối ngày

• textcnn_agnews.ipynb

• textcnn.py

• tfidf_vs_textcnn_errors.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Thêm pretrained/static embedding từ ngày 9 và đo có giúp không.

NGÀY 11 - RNN, LSTM, GRU: sequence modeling

Dataset	IMDb subset
Cách lấy dữ liệu	Hugging Face IMDb; chọn 15k-25k train để đủ sequence signal.
Metric chính	Validation F1 + epoch time + stability của loss.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Hiểu hidden state, vanishing gradient và tự dùng LSTM/GRU cho sequence classification.

1. Học lý thuyết gì?

• RNN recurrence; hidden state.

• Vanishing/exploding gradient.

• LSTM gates (input/forget/output) ở mức hiểu flow; GRU đơn giản hơn.

• last hidden vs mean/max pooling over sequence.

• Bidirectional concept.

2. Triển khai từng bước

1. Tái dùng vocabulary/DataLoader.

2. Implement RNN baseline trước 1 epoch để hiểu shape.

3. Implement LSTM và GRU; batch_first=True.

4. Mask/pad đúng; thử last hidden và mean pooling.

5. Gradient clipping (clip_grad_norm_).

6. So LSTM vs GRU runtime/F1.

3. Experiments bắt buộc

• E01 vanilla RNN.

• E02 LSTM.

• E03 GRU.

• E04 GRU/LSTM + bidirectional nếu thời gian.

4. Tiêu chí PASS

PASS: Bạn vẽ được shape [B,L,E] -> recurrent outputs và giải thích h_n; code có gradient clipping và best checkpoint.

5. Nguồn học lý thuyết

• PyTorch Sequence Models/LSTM - Tutorial chính thức.

• CS224N - RNN/LSTM lecture/notes.

6. Nguồn tham khảo code / API

• PyTorch sequence model code - Code LSTM.

• IMDb dataset - Dataset.

7. Lỗi hay mắc

• Lấy output ở PAD timestep làm representation.

• Không clip gradient khi loss bùng.

• Train sequence quá dài trước khi có baseline chạy.

8. Deliverables cuối ngày

• rnn_lstm_gru.ipynb

• sequence_models.py

• runtime_comparison.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Thử pack_padded_sequence/pad_packed_sequence và đo speedup.

NGÀY 12 - BiLSTM, padding/packing và classifier tốt hơn

Dataset	IMDb
Cách lấy dữ liệu	Dùng split cố định ngày 11 để so công bằng.
Metric chính	F1 + samples/sec + average padding ratio.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Nắm cách xử lý variable-length sequence chính xác và xây BiLSTM classifier có mask/packing.

1. Học lý thuyết gì?

• Bidirectional LSTM: forward/backward contexts.

• Padding, attention mask kiểu recurrent, packed sequences.

• Pooling: last states, masked mean, max.

• Dropout giữa recurrent/classifier layers.

2. Triển khai từng bước

1. Viết collate_fn trả lengths.

2. sort/enforce_sorted=False + pack_padded_sequence.

3. BiLSTM -> concat final states hoặc masked pooling.

4. Đo batch throughput và F1 với/không packing.

5. Kiểm tra một batch bằng assert shapes/lengths.

3. Experiments bắt buộc

• E01 unidirectional LSTM.

• E02 BiLSTM.

• E03 BiLSTM + packing.

• E04 different pooling.

4. Tiêu chí PASS

PASS: Không có bug PAD; output ổn định; bạn giải thích được packing có lợi khi length variance lớn.

5. Nguồn học lý thuyết

• PyTorch LSTM tutorial - Sequence models.

• PyTorch packed sequence docs - Packing API.

6. Nguồn tham khảo code / API

• PyTorch DataLoader - collate pattern.

• IMDb dataset - Dataset.

7. Lỗi hay mắc

• Sort lengths nhưng quên unsort prediction.

• Nhầm h_n dimensions khi bidirectional.

• Padding toàn dataset tới max tuyệt đối.

8. Deliverables cuối ngày

• bilstm_packing.ipynb

• collate.py

• packing_benchmark.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Thử bucket batches theo length để giảm padding.

NGÀY 13 - Attention từ scratch: Q, K, V

Dataset	AG News subset
Cách lấy dữ liệu	Lấy 5k-10k samples; mục tiêu debug attention, không chasing score.
Metric chính	Correctness/shape/mask là chính; metric classification là phụ.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Tự implement scaled dot-product attention và hiểu tensor shapes/masks.

1. Học lý thuyết gì?

• Q/K/V trực giác: query tìm thông tin, key để match, value là nội dung.

• Scaled dot-product attention.

• softmax dimension.

• padding mask; self-attention.

• multi-head: nhiều subspaces.

2. Triển khai từng bước

1. Trên tensor toy [B,L,D], tự tạo Wq/Wk/Wv.

2. Tính scores=QK^T/sqrt(dk); apply mask trước softmax.

3. Kiểm tra row sum attention≈1.

4. Tạo SelfAttentionClassifier: embedding -> attention -> masked mean -> linear.

5. So với mean-embedding baseline ngày 8.

6. Visualize attention cho 5 câu chỉ để debug.

3. Experiments bắt buộc

• E01 no attention mean pooling.

• E02 single-head self-attention.

• E03 4-head bằng nn.MultiheadAttention.

4. Tiêu chí PASS

PASS: Bạn tự ghi được công thức, dự đoán đúng shape ở từng dòng và giải thích padding mask; code pass shape tests.

5. Nguồn học lý thuyết

• CS224N Attention/Transformers - Lecture attention/transformers.

• PyTorch MultiheadAttention - Reference implementation + shapes.

6. Nguồn tham khảo code / API

• MultiheadAttention docs - API và output weights.

• AG News dataset - Dataset nhỏ cho lab.

7. Lỗi hay mắc

• Softmax nhầm dimension.

• Mask sau softmax thay vì trước.

• Quên sqrt(dk).

8. Deliverables cuối ngày

• attention_from_scratch.ipynb

• attention.py

• shape_tests.py

9. Stretch goal (chỉ làm sau khi PASS)

• Viết unit test so output attention tự code với torch.scaled_dot_product_attention trên tensor nhỏ.

NGÀY 14 - Mini Transformer Encoder classifier

Dataset	AG News subset
Cách lấy dữ liệu	Dùng 20k samples; vocabulary tự xây, không pretrained.
Metric chính	Macro-F1/Accuracy + params + runtime; trọng tâm kiến trúc.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Ghép embedding + positional encoding + TransformerEncoder thành classifier; hiểu lý do pretrained Transformer sẽ mạnh hơn.

1. Học lý thuyết gì?

• Positional encoding.

• Multi-head self-attention + residual + LayerNorm + FFN.

• Encoder stack.

• [CLS]-like learned token vs masked mean pooling.

• Pretraining vs training from scratch.

2. Triển khai từng bước

1. Token IDs + padding mask.

2. Embedding + learned/sinusoidal positional encoding.

3. nn.TransformerEncoderLayer(batch_first=True).

4. 2 encoder layers, 4 heads, d_model 128/256.

5. masked mean -> Linear classifier.

6. Train ngắn và so với TextCNN/LSTM.

3. Experiments bắt buộc

• E01 1 layer.

• E02 2 layers.

• E03 d_model 128 vs 256 nếu GPU cho phép.

4. Tiêu chí PASS

PASS: Pipeline chạy được và bạn giải thích được mọi block; nhận ra from-scratch cần nhiều data/compute và không cố tune quá mức.

5. Nguồn học lý thuyết

• CS224N Transformer materials - Kiến trúc Transformer.

• PyTorch MultiheadAttention - Attention building block.

6. Nguồn tham khảo code / API

• PyTorch TransformerEncoder docs - Encoder API.

• AG News - Dataset.

7. Lỗi hay mắc

• Quên positional information.

• Padding mask sai polarity/shape.

• Model quá lớn khiến cả ngày chỉ train 1 run.

8. Deliverables cuối ngày

• mini_transformer.ipynb

• transformer_encoder.py

• week2_model_comparison.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Tạo bảng “TF-IDF vs TextCNN vs LSTM vs mini Transformer”: metric, train time, params.

NGÀY 15 - BERT/Transformer pretrained và tokenizer

Dataset	SST-2 (GLUE)
Cách lấy dữ liệu	Hugging Face: load_dataset("nyu-mll/glue", "sst2")
Metric chính	Không cần fine-tune; đo truncation rate, tokens/sample, inference shape.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Hiểu pretrained encoder và tokenizer; làm inference, inspect token IDs/masks trước khi fine-tune.

1. Học lý thuyết gì?

• BERT encoder-only; MLM pretraining; fine-tuning.

• WordPiece/BPE/subword; special tokens.

• input_ids, attention_mask, token_type_ids (khi model dùng).

• CLS representation/logits.

• Truncation và max_length.

2. Triển khai từng bước

1. Load AutoTokenizer + AutoModelForSequenceClassification pretrained.

2. Tokenize 10 câu, decode từng token.

3. So tokenization của từ thường, rare, typo, punctuation.

4. Batch tokenize với padding/truncation.

5. Inference no_grad; inspect logits/softmax.

6. Đo length distribution theo tokenizer để chọn max_length hợp lý.

3. Experiments bắt buộc

• E01 max_length 64/128 chỉ đo truncation rate.

• E02 static padding=max_length vs dynamic padding, chưa train.

4. Tiêu chí PASS

PASS: Bạn nhìn batch dict và biết từng tensor là gì; chọn max_length dựa p95/p99 token length thay vì 512 mặc định.

5. Nguồn học lý thuyết

• Hugging Face Course - Models/tokenizers basics.

• Padding & truncation - Chiến lược padding/truncation.

6. Nguồn tham khảo code / API

• Text classification guide - AutoTokenizer + AutoModelForSequenceClassification.

• SST-2 dataset - Dataset.

7. Lỗi hay mắc

• Dùng max_length=512 cho mọi dataset.

• Nhầm tokenizer token count với word count.

• Softmax rồi dùng CrossEntropyLoss trên probabilities trong custom loop.

8. Deliverables cuối ngày

• bert_tokenizer_lab.ipynb

• token_length_report.csv

• bert_cheatsheet.md

9. Stretch goal (chỉ làm sau khi PASS)

• So tokenizer BERT/DistilBERT/XLM-R trên 20 câu chứa typo/tên riêng.

NGÀY 16 - Hugging Face Datasets + Trainer: pipeline chuẩn

Dataset	IMDb
Cách lấy dữ liệu	Hugging Face IMDb; dùng 10k-20k train subset để iteration.
Metric chính	Validation accuracy/F1 + GPU memory + samples/sec.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Tự nối Dataset.map -> tokenizer -> dynamic padding -> compute_metrics -> Trainer và hiểu từng thành phần.

1. Học lý thuyết gì?

• datasets.Dataset.map/batched.

• DataCollatorWithPadding.

• TrainingArguments và Trainer.

• compute_metrics.

• save/evaluate/predict.

2. Triển khai từng bước

1. Load dataset và tạo validation từ train hoặc dùng split hợp lý.

2. Tokenizer map batched, không padding max_length cứng.

3. DataCollatorWithPadding.

4. AutoModelForSequenceClassification DistilBERT.

5. Trainer 1 epoch; compute accuracy + F1.

6. trainer.predict và tạo DataFrame errors.

3. Experiments bắt buộc

• E01 max_length 128 dynamic padding.

• E02 256 nếu p95 cần.

• E03 batch size phù hợp GPU.

4. Tiêu chí PASS

PASS: Notebook từ raw dataset đến prediction chạy một mạch; bạn biết chỗ nào diễn ra tokenization, collate, forward, metric.

5. Nguồn học lý thuyết

• HF Fine-tuning with Trainer - Trainer workflow.

• HF Data collators - Dynamic padding.

6. Nguồn tham khảo code / API

• HF Text Classification guide - End-to-end code.

• IMDb dataset - Dataset.

7. Lỗi hay mắc

• Map tokenizer rồi giữ cột string không cần thiết gây RAM.

• Padding global max.

• Không set load_best_model_at_end/eval strategy phù hợp.

8. Deliverables cuối ngày

• hf_trainer_imdb.ipynb

• trainer_config.json

• errors.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Viết phiên bản custom PyTorch training loop cho cùng model để hiểu Trainer che giấu gì.

NGÀY 17 - Fine-tune Transformer cho multiclass

Dataset	AG News
Cách lấy dữ liệu	load_dataset("fancyzhx/ag_news")
Metric chính	Accuracy/Macro-F1 + minutes/run + best epoch.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Fine-tune DistilBERT/RoBERTa-like model cho 4 class; tạo baseline pretrained đáng tin cậy.

1. Học lý thuyết gì?

• Sequence classification head.

• Learning rate nhỏ khi fine-tuning.

• Warmup/weight decay ở mức thực dụng.

• Overfitting signal qua learning curves.

2. Triển khai từng bước

1. Baseline TF-IDF ngày 7 để làm mốc.

2. Fine-tune distilbert-base-uncased 2-3 epochs, LR 2e-5.

3. Log train/eval loss mỗi epoch.

4. Save best model by macro-F1/accuracy.

5. Error analysis 50 samples; xem confidence.

6. So speed/score với classical baseline.

3. Experiments bắt buộc

• E01 LR 2e-5, max_len128.

• E02 LR 3e-5.

• E03 max_len 64 nếu text ngắn.

4. Tiêu chí PASS

PASS: Có Transformer baseline reproducible; biết khi nào điểm tăng đáng chi phí so với TF-IDF.

5. Nguồn học lý thuyết

• HF Text Classification guide - Sequence classification fine-tuning.

• HF Fine-tuning chapter - Trainer + evaluation.

6. Nguồn tham khảo code / API

• AG News dataset - Dataset.

• Transformers Trainer docs - Trainer API.

7. Lỗi hay mắc

• Tune 20 combinations.

• Đổi model và LR cùng lúc rồi không biết nguyên nhân.

• Chọn checkpoint theo train loss.

8. Deliverables cuối ngày

• agnews_transformer.ipynb

• learning_curve.csv

• classical_vs_transformer.md

9. Stretch goal (chỉ làm sau khi PASS)

• Thử model nhỏ hơn để tìm Pareto score/speed tốt nhất cho contest.

NGÀY 18 - NLP tiếng Việt với PhoBERT và XLM-R

Dataset	UIT-VSFC
Cách lấy dữ liệu	GitHub UIT-VSFC; chuẩn bị raw và VnCoreNLP segmented variants.
Metric chính	Macro-F1 + per-class F1 + runtime + truncation rate.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Xây pipeline tiếng Việt đúng tokenizer/model và biết khi nào dùng PhoBERT vs multilingual XLM-R.

1. Học lý thuyết gì?

• PhoBERT dựa RoBERTa và khuyến nghị word segmentation.

• XLM-R multilingual không yêu cầu cùng pipeline segmentation như PhoBERT.

• Vietnamese Unicode/tone normalization.

• Domain mismatch và tokenizer fragmentation.

2. Triển khai từng bước

1. Tạo split stratified cố định.

2. Fine-tune PhoBERT trên word-segmented text.

3. Fine-tune XLM-R trên raw/light-normalized text.

4. Đo token length/truncation rate cho cả hai.

5. So macro-F1, runtime, confusion matrix.

6. Đọc 50 lỗi và phân loại: negation, slang, topic ambiguity, very short feedback.

3. Experiments bắt buộc

• E01 PhoBERT base + segmented.

• E02 XLM-R base + raw.

• E03 PhoBERT raw (để chứng minh impact segmentation) nếu đủ thời gian.

4. Tiêu chí PASS

PASS: Có pipeline tiếng Việt từ raw -> segmentation/tokenizer -> model; không trộn preprocessing giữa model mà không ghi rõ.

5. Nguồn học lý thuyết

• PhoBERT official - Usage + segmentation notes.

• HF Text Classification - Fine-tuning pattern.

6. Nguồn tham khảo code / API

• UIT-VSFC - Dataset.

• PhoBERT official code - Tokenizer/model usage.

7. Lỗi hay mắc

• Remove dấu/stopword cho Transformer.

• Dùng PhoBERT nhưng không word-segment rồi kết luận model yếu.

• So model trên split khác nhau.

8. Deliverables cuối ngày

• vietnamese_phobert_xlmr.ipynb

• segmented_cache/

• vi_model_comparison.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Tạo char-TF-IDF LinearSVC baseline tiếng Việt và ensemble với Transformer nếu errors bổ sung nhau.

NGÀY 19 - Hyperparameter tuning có kỷ luật + experiment tracking

Dataset	UIT-VSFC
Cách lấy dữ liệu	Giữ dataset/split ngày 18; chọn model thắng hoặc rẻ hơn.
Metric chính	Macro-F1 mean over 2 seeds + runtime.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Học cách tune trong contest: ít run nhưng nhiều thông tin; biết thứ tự ưu tiên LR -> length -> batch/effective batch -> epoch.

1. Học lý thuyết gì?

• Learning rate/epochs/batch size/max length.

• Effective batch size và gradient accumulation.

• Early stopping/load best model.

• Seed variance.

• One-factor-at-a-time vs small factorial search.

2. Triển khai từng bước

1. Baseline config từ ngày 18.

2. Run LR 1e-5/2e-5/3e-5 với mọi thứ giữ nguyên.

3. Chọn LR tốt; thử max_len 128/256 dựa truncation.

4. Batch lớn nhất GPU chứa; nếu không đủ dùng accumulation.

5. Run 2 seeds cho config tốt nhất.

6. Tạo experiment table có delta metric + delta runtime.

3. Experiments bắt buộc

• E01/E02/E03 LR sweep.

• E04 max_length.

• E05 effective batch size.

• E06 second seed best config.

4. Tiêu chí PASS

PASS: Bạn dừng tune khi gain nhỏ so thời gian; experiment log đủ để người khác tái hiện.

5. Nguồn học lý thuyết

• HF TrainingArguments/Trainer - Fine-tuning controls.

• HF gradient accumulation - Effective batch size.

6. Nguồn tham khảo code / API

• HF GPU training efficiency - Runtime/memory techniques.

• PhoBERT - Model reference.

7. Lỗi hay mắc

• Grid search mù.

• Mỗi run đổi 4 thứ.

• Không ghi code/model revision/seed.

8. Deliverables cuối ngày

• experiments_day19.csv

• best_config.json

• tuning_summary.md

9. Stretch goal (chỉ làm sau khi PASS)

• Tạo Pareto chart metric vs train_minutes từ experiment log.

NGÀY 20 - Multi-label classification, imbalance và threshold tuning

Dataset	Jigsaw Toxic Comment Classification
Cách lấy dữ liệu	Kaggle: train.csv có comment_text và 6 nhãn toxicity.
Metric chính	Micro-F1 + Macro-F1 hoặc ROC-AUC tùy lab; theo dõi rare labels.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Phân biệt multiclass vs multilabel; dùng BCEWithLogitsLoss/ sigmoid và tune threshold đúng cách.

1. Học lý thuyết gì?

• Multiclass: một nhãn; multilabel: nhiều nhãn độc lập.

• sigmoid vs softmax.

• BCEWithLogitsLoss và pos_weight.

• Micro/macro F1 cho multilabel; ROC-AUC nếu đề dùng.

• Per-class threshold tuning trên validation.

2. Triển khai từng bước

1. Lấy subset 30k-80k để chạy trong ngày.

2. EDA positive rate từng label.

3. TF-IDF + OneVsRest Logistic baseline.

4. Transformer AutoModelForSequenceClassification(num_labels=6, problem_type="multi_label_classification").

5. Tune global threshold 0.3-0.7 trên validation; sau đó per-class threshold nếu cần.

6. So micro/macro F1 và rare-class recall.

3. Experiments bắt buộc

• E01 threshold 0.5.

• E02 global threshold optimized.

• E03 pos_weight/weighted loss.

• E04 per-class thresholds.

4. Tiêu chí PASS

PASS: Không dùng argmax; submission có 6 probabilities/labels đúng format; threshold chỉ tune validation.

5. Nguồn học lý thuyết

• Jigsaw dataset - Dataset description + labels.

• HF text classification - Transformer classification base pattern.

6. Nguồn tham khảo code / API

• BCEWithLogitsLoss docs - pos_weight + logits.

• Jigsaw data - Columns/sample submission.

7. Lỗi hay mắc

• Softmax cho multilabel.

• Argmax lấy 1 class.

• Tuning threshold trên public leaderboard.

8. Deliverables cuối ngày

• jigsaw_multilabel.ipynb

• thresholds.json

• per_label_metrics.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Thử char-TF-IDF baseline cho toxic obfuscation/typo và xem ensemble có bổ sung Transformer không.

NGÀY 21 - Mini Contest #2 - Dataset lạ, classical vs Transformer

Dataset	dair-ai/emotion (dùng split/unsplit khác chưa xem sâu hoặc thay bằng dataset do teammate chọn)
Cách lấy dữ liệu	Nếu đã dùng emotion nhiều ở ngày 6, teammate chọn DBpedia/TREC/Yelp để giữ độ lạ.
Metric chính	Metric do task quy định; nếu multiclass dùng Macro-F1 + Accuracy.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Trong 3 giờ phải quyết định nhanh baseline nào, model pretrained nào, metric nào và experiment tiếp theo.

1. Học lý thuyết gì?

• Không học mới. Ôn timeboxing, baseline-first, experiment log và error analysis.

2. Triển khai từng bước

1. 0:00-0:20: read/EDA/split/metric.

2. 0:20-0:40: TF-IDF + LinearSVC.

3. 0:40-1:50: pretrained Transformer run đầu.

4. 1:50-2:25: error analysis + đúng 1-2 cải tiến.

5. 2:25-2:50: ensemble/tune nhẹ nếu validation chứng minh.

6. 2:50-3:00: clean output + reproducibility check.

3. Experiments bắt buộc

• Classical baseline.

• Transformer baseline.

• 1 preprocessing/length/LR experiment.

• Optional ensemble.

4. Tiêu chí PASS

PASS: PASS nếu baseline <=20m; Transformer <=90m; không có data leakage; final run/submission được lưu.

5. Nguồn học lý thuyết

• HF Text Classification - Documentation được phép tra.

• scikit-learn Text Data - Classical baseline docs.

6. Nguồn tham khảo code / API

• Emotion dataset - Nếu dùng dataset này.

• HF datasets loading - Load/slicing.

7. Lỗi hay mắc

• Mất 45 phút chọn model.

• Không đặt metric trước khi tune.

• Dùng model cực lớn khiến chỉ có 1 run.

8. Deliverables cuối ngày

• contest_day21.ipynb

• experiments.csv

• submission_or_predictions.csv

• postmortem.md

9. Stretch goal (chỉ làm sau khi PASS)

• Sau contest viết 5 phút: nếu thêm 30 phút, experiment nào có expected value cao nhất và vì sao?

NGÀY 22 - Token Classification / Named Entity Recognition (NER)

Dataset	CoNLL-2003 (tner/conll2003)
Cách lấy dữ liệu	Hugging Face dataset; tokens + tags; 4 entity types ORG/PER/LOC/MISC.
Metric chính	Seqeval entity-level precision/recall/F1.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết xử lý bài gán nhãn từng token: BIO tags, subword-label alignment và seqeval.

1. Học lý thuyết gì?

• Sequence vs token classification.

• BIO/IOB2 tagging.

• Tokenizer split một word thành nhiều subwords.

• word_ids() và label alignment; -100 để ignore special/subword tokens theo strategy.

• Entity-level precision/recall/F1 (seqeval).

2. Triển khai từng bước

1. Load tokens/tags và map id2label.

2. Inspect 20 examples BIO.

3. Tokenizer is_split_into_words=True.

4. Viết tokenize_and_align_labels().

5. AutoModelForTokenClassification.

6. DataCollatorForTokenClassification + Trainer.

7. Evaluate seqeval; inspect entity errors.

3. Experiments bắt buộc

• E01 DistilBERT/BERT token classification.

• E02 max_length/truncation handling.

• E03 first-subword-only vs all-subwords label strategy (nếu hiểu rõ).

4. Tiêu chí PASS

PASS: Bạn xử lý đúng alignment; không tính special/PAD tokens vào loss; báo cáo entity-level F1.

5. Nguồn học lý thuyết

• HF Token Classification Guide - End-to-end NER fine-tune.

• CoNLL-2003 dataset - BIO tags và splits.

6. Nguồn tham khảo code / API

• HF Token Classification code - tokenize_and_align_labels, data collator.

• CoNLL dataset card - Label mapping.

7. Lỗi hay mắc

• Gán cùng label sai cho mọi subword mà không hiểu strategy.

• Dùng accuracy token-level vì class O áp đảo.

• Không inspect truncation cắt entity.

8. Deliverables cuối ngày

• ner_conll.ipynb

• label_alignment_tests.py

• ner_errors.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Nếu còn giờ, thử một multilingual model để chuẩn bị NER tiếng Việt trong tương lai.

NGÀY 23 - Sentence-pair classification: paraphrase/NLI pattern

Dataset	GLUE QQP (Quora Question Pairs)
Cách lấy dữ liệu	load_dataset("nyu-mll/glue", "qqp"); dùng subset 50k-100k để chạy nhanh.
Metric chính	F1/Accuracy theo lab; theo dõi positive-class recall.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết tokenize hai câu cùng lúc và xử lý bài "quan hệ giữa hai câu", không nối text thủ công vô tội vạ.

1. Học lý thuyết gì?

• Paired inputs tokenizer(sentence1, sentence2).

• Cross-encoder representation cho pair classification.

• Class imbalance/duplicate pairs.

• Pair order invariance: paraphrase thường symmetric nhưng model input có order.

2. Triển khai từng bước

1. Inspect question1/question2/label.

2. Classical baseline: TF-IDF riêng/concat hoặc cosine features đơn giản.

3. Transformer tokenizer(q1,q2,truncation=True).

4. Fine-tune sequence classifier.

5. Test swap(q1,q2) 1000 samples để xem prediction consistency.

6. Error analysis: lexical overlap high nhưng non-duplicate, low overlap nhưng duplicate.

3. Experiments bắt buộc

• E01 TF-IDF cosine + Logistic.

• E02 Transformer cross-encoder.

• E03 pair swap augmentation hoặc consistency check.

4. Tiêu chí PASS

PASS: Bạn tạo batch paired inputs đúng và giải thích cross-encoder khác single sentence classifier.

5. Nguồn học lý thuyết

• GLUE dataset card - QQP task nằm trong GLUE.

• HF text classification - Tokenizer/model sequence classification.

6. Nguồn tham khảo code / API

• QQP files - Dataset.

• Tokenizer padding/truncation - Pair truncation strategies.

7. Lỗi hay mắc

• Nối “q1 + q2” mà không separator/tokenizer pair API.

• Random split nếu task có group leakage mà không kiểm tra.

• Chỉ nhìn lexical overlap.

8. Deliverables cuối ngày

• qqp_sentence_pair.ipynb

• swap_consistency.csv

• pair_error_analysis.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Thử truncation="longest_first" vs only_second trên pair dài.

NGÀY 24 - Semantic similarity / regression với sentence pairs

Dataset	GLUE STS-B
Cách lấy dữ liệu	load_dataset("nyu-mll/glue", "stsb"); label 0-5 similarity.
Metric chính	Pearson + Spearman + MSE.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết bài NLP có output liên tục; xử lý regression và correlation metric thay vì ép thành classification.

1. Học lý thuyết gì?

• Semantic textual similarity.

• Regression head num_labels=1/problem type regression.

• MSE loss; Pearson/Spearman correlation.

• Cross-encoder vs bi-encoder concept (chỉ cần hiểu).

2. Triển khai từng bước

1. EDA label distribution 0-5.

2. Classical: TF-IDF cosine as baseline score; fit simple regressor nếu muốn.

3. Transformer pair tokenizer; sequence regression head.

4. Evaluate Pearson + Spearman + MSE.

5. Plot/inspect 30 largest absolute errors.

6. Phân nhóm: negation, numbers, entities, lexical overlap.

3. Experiments bắt buộc

• E01 TF-IDF cosine baseline.

• E02 pretrained cross-encoder fine-tune.

• E03 max_length/LR small change.

4. Tiêu chí PASS

PASS: Bạn không threshold label thành class; biết chọn metric correlation và đọc error continuous.

5. Nguồn học lý thuyết

• GLUE STS-B dataset - Pair + float label.

• CS224N - Sentence representation/semantic similarity context.

6. Nguồn tham khảo code / API

• STS-B dataset viewer - Examples.

• HF sequence classification - Dùng AutoModelForSequenceClassification với regression config.

7. Lỗi hay mắc

• Ép 0-5 thành 6 classes.

• Chỉ báo MSE khi task đánh Spearman/Pearson.

• Dùng cosine TF-IDF như ground truth thay vì baseline.

8. Deliverables cuối ngày

• stsb_similarity.ipynb

• regression_errors.csv

• metric_notes.md

9. Stretch goal (chỉ làm sau khi PASS)

• Khảo sát Sentence Transformers như bi-encoder sau contest roadmap, không để lệch stack chính hôm nay.

NGÀY 25 - Noisy text, augmentation và robustness

Dataset	Kaggle Disaster Tweets
Cách lấy dữ liệu	train.csv: text + keyword + location + binary target.
Metric chính	F1/Accuracy + robustness delta trên corrupted validation.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết xử lý noisy social text và chỉ dùng augmentation khi validation chứng minh; tận dụng metadata nếu hợp lý.

1. Học lý thuyết gì?

• Noise: hashtag, URL, mention, misspelling, capitalization.

• Word/char features thường rất mạnh với noisy short text.

• Augmentation risks: semantic drift, label corruption.

• Metadata fusion ở mức đơn giản.

2. Triển khai từng bước

1. EDA text/keyword/location missingness.

2. TF-IDF word + char LinearSVC baseline.

3. Transformer baseline.

4. Thử LIGHT normalization (URL token, whitespace).

5. Thử một augmentation rất nhẹ trên minority/hard cases; không làm 5 loại cùng lúc.

6. Optional: thêm keyword feature vào classical/transformer text template.

3. Experiments bắt buộc

• E01 raw classical.

• E02 light-clean classical.

• E03 transformer raw/light.

• E04 augmentation only if reasoned.

4. Tiêu chí PASS

PASS: Bạn có bằng chứng augmentation giúp/hại; không "clean social text" đến mức mất hashtag/keyword signal.

5. Nguồn học lý thuyết

• Disaster Tweets dataset - Data format + sample submission.

• TfidfVectorizer char/word - Noisy text features.

6. Nguồn tham khảo code / API

• Kaggle data page - Dataset.

• HF text classification - Transformer baseline reference.

7. Lỗi hay mắc

• Remove #/@/URL một cách mù quáng.

• Back-translation hàng nghìn câu trước khi baseline.

• Leak location/keyword processing across split.

8. Deliverables cuối ngày

• noisy_text_lab.ipynb

• augmentation_ablation.csv

• robustness_errors.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Tự tạo corruption test: typo, repeated chars, URL replacement; đo performance drop của classical vs Transformer.

NGÀY 26 - Validation nâng cao, K-Fold, OOF và data leakage

Dataset	GLUE QQP subset hoặc dataset có duplicate/groups
Cách lấy dữ liệu	Dùng QQP vì cùng câu hỏi có thể xuất hiện nhiều cặp; tạo question graph/group đơn giản để suy nghĩ về leakage.
Metric chính	CV mean/std; quan trọng nhất là score gap random vs group-aware.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết khi random stratified split chưa đủ và dùng group-aware reasoning; tạo OOF predictions cho stacking/ensemble.

1. Học lý thuyết gì?

• Leakage: exact/near duplicate, same user/entity/group, time leakage.

• StratifiedKFold vs GroupKFold/StratifiedGroupKFold.

• OOF prediction.

• CV mean/std và cost trade-off trong contest.

2. Triển khai từng bước

1. Tìm exact duplicated pairs và repeated question IDs/texts.

2. Random stratified split baseline.

3. Tạo group ID đơn giản từ connected question IDs hoặc dùng one-side question ID khi khả thi.

4. So random vs group-aware score; giải thích gap.

5. Trên dataset nhỏ, chạy 3-fold TF-IDF để tạo OOF.

6. Lưu fold assignment để reproducible.

3. Experiments bắt buộc

• E01 single stratified split.

• E02 3-fold stratified.

• E03 group-aware split.

4. Tiêu chí PASS

PASS: Bạn nhận ra split có thể làm điểm "ảo"; biết tạo OOF và không fit vectorizer ngoài fold.

5. Nguồn học lý thuyết

• StratifiedKFold docs - CV classification.

• GroupKFold docs - Group separation.

6. Nguồn tham khảo code / API

• QQP dataset - Repeated questions/pairs.

• scikit Pipeline docs - Fit preprocessing inside fold.

7. Lỗi hay mắc

• Fit TF-IDF trên full data rồi CV classifier.

• KFold Transformer 5 folds trong contest 6h mà không tính compute.

• Không lưu fold indices.

8. Deliverables cuối ngày

• validation_leakage.ipynb

• folds.csv

• random_vs_group_score.md

9. Stretch goal (chỉ làm sau khi PASS)

• Viết checklist 10 leakage questions áp dụng ngay khi nhận đề OlpAI.

NGÀY 27 - Ensemble, calibration và chọn model theo expected gain

Dataset	AG News hoặc UIT-VSFC
Cách lấy dữ liệu	Tái dùng 2-3 model đã train để tránh mất ngày chỉ train mới.
Metric chính	Validation metric gain over best single + extra cost.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Biết ensemble khi model lỗi khác nhau; tránh ensemble mù và hiểu confidence/probability cơ bản.

1. Học lý thuyết gì?

• Hard voting vs soft voting.

• SVM decision score vs calibrated probability.

• Diversity/error overlap.

• OOF tuning ensemble weights để tránh test/leaderboard overfit.

2. Triển khai từng bước

1. Chuẩn bị prediction validation của TF-IDF, Transformer A, Transformer B.

2. Tính error overlap/Jaccard giữa models.

3. Soft average 0.5/0.5, rồi 0.3/0.7 trên validation.

4. Nếu LinearSVC cần probability: dùng calibrated model hoặc normalize scores cẩn thận; không giả decision_function là probability.

5. Đo gain ensemble và thêm inference cost.

6. Chỉ giữ ensemble nếu gain đáng kể.

3. Experiments bắt buộc

• E01 best single.

• E02 50/50.

• E03 weight sweep nhỏ.

• E04 3-model ensemble nếu diversity tốt.

4. Tiêu chí PASS

PASS: Bạn chứng minh ensemble tăng validation; có weight/config lưu lại và biết cost tăng bao nhiêu.

5. Nguồn học lý thuyết

• scikit calibration - Probability calibration.

• HF text classification - Transformer probabilities/logits.

6. Nguồn tham khảo code / API

• CalibratedClassifierCV docs - SVM calibration.

• AG News - Dataset reference.

7. Lỗi hay mắc

• Ensemble 3 model cùng kiến trúc/seed với errors giống nhau.

• Tune weight trên test.

• Trộn logits khác scale như probabilities.

8. Deliverables cuối ngày

• ensemble_lab.ipynb

• ensemble_weights.json

• error_overlap.csv

9. Stretch goal (chỉ làm sau khi PASS)

• Vẽ gain vs inference/training cost để quyết định chiến thuật contest.

NGÀY 28 - Tối ưu GPU và tốc độ fine-tuning

Dataset	AG News hoặc UIT-VSFC
Cách lấy dữ liệu	Chọn Transformer baseline đã ổn để benchmark apples-to-apples.
Metric chính	Metric + train minutes + max VRAM + samples/sec.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Giảm thời gian/VRAM mà không làm metric giảm đáng kể; chuẩn bị cho tài nguyên thi giới hạn.

1. Học lý thuyết gì?

• Dynamic padding.

• Mixed precision fp16/bf16.

• Gradient accumulation và effective batch size.

• Gradient checkpointing trade-off.

• Freeze layers/early stopping.

• Caching tokenized dataset.

2. Triển khai từng bước

1. Baseline đo GPU memory + samples/sec + total train time.

2. Bật DataCollatorWithPadding.

3. Mixed precision nếu GPU hỗ trợ.

4. Tìm per_device_batch_size lớn nhất fit; dùng accumulation chỉ khi cần.

5. Thử freeze embeddings/low layers trong một run ngắn.

6. Ghi bảng score/time/VRAM.

3. Experiments bắt buộc

• E01 baseline FP32/static padding nếu có.

• E02 dynamic padding.

• E03 mixed precision.

• E04 accumulation / checkpointing khi thiếu VRAM.

• E05 freeze layers.

4. Tiêu chí PASS

PASS: Có config "contest-fast" với metric giảm rất nhỏ nhưng thời gian/VRAM tốt hơn; biết kỹ thuật nào làm chậm dù tiết kiệm memory.

5. Nguồn học lý thuyết

• HF GPU efficiency - Mixed precision/checkpointing/accumulation.

• HF Data Collators - Dynamic padding.

6. Nguồn tham khảo code / API

• HF Gradient accumulation - Effective batch size.

• Padding/truncation - Input efficiency.

7. Lỗi hay mắc

• Bật mọi optimization cùng lúc.

• Gradient accumulation quá nhiều làm throughput kém.

• Không benchmark cùng dataset/model/seed.

8. Deliverables cuối ngày

• gpu_optimization.ipynb

• benchmark_gpu.csv

• contest_fast_config.json

9. Stretch goal (chỉ làm sau khi PASS)

• Tạo script estimate_runtime.py: từ samples/sec ước lượng thời gian train toàn dataset trước khi bấm chạy.

NGÀY 29 - Mock Contest #1 - Text classification/noisy data

Dataset	Kaggle Disaster Tweets (làm lại từ notebook trắng, không xem ngày 25)
Cách lấy dữ liệu	Dùng train/test/sample_submission đúng kiểu competition.
Metric chính	F1 (theo competition) + time-to-first-valid-model.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Mô phỏng 4 giờ: baseline nhanh, Transformer, error analysis, optional ensemble và submission sanity check.

1. Học lý thuyết gì?

• Không học mới. Chỉ dùng documentation chính thức nếu quên API.

2. Triển khai từng bước

1. 0:00-0:15 đọc đề/data/metric.

2. 0:15-0:35 split + TF-IDF baseline.

3. 0:35-1:45 Transformer first run.

4. 1:45-2:15 error analysis + metadata/cleaning decision.

5. 2:15-3:10 one improvement + optional second model.

6. 3:10-3:35 ensemble nếu validation có gain.

7. 3:35-4:00 retrain/save/submission checks.

3. Experiments bắt buộc

• Baseline word+char.

• Transformer.

• 1 targeted improvement.

• Optional ensemble.

4. Tiêu chí PASS

PASS: PASS: baseline <=20m; final prediction không NaN, đúng row count/order; notebook có thể chạy lại; postmortem nêu 3 quyết định đúng/sai.

5. Nguồn học lý thuyết

• Disaster Tweets - Competition data.

• HF text classification - Transformer docs.

6. Nguồn tham khảo code / API

• scikit text pipeline - Classical reference.

• Kaggle sample submission - Kiểm format.

7. Lỗi hay mắc

• Xem notebook top Kaggle trước khi thi mock.

• Submission row order sai.

• Tuning theo leaderboard thay validation.

8. Deliverables cuối ngày

• mock29.ipynb

• submission.csv

• experiments.csv

• postmortem.md

9. Stretch goal (chỉ làm sau khi PASS)

• Lặp lại chỉ 2.5 giờ vào ngày nghỉ nếu vẫn mất >25m để có baseline.

NGÀY 30 - FINAL MOCK - NLP task bí mật + tổng kết

Dataset	Teammate chọn một dataset bạn chưa làm: TREC/DBpedia/Yelp/NER/QQP/STSB
Cách lấy dữ liệu	Bạn chỉ biết file/schema lúc bắt đầu. Nếu không có teammate, bốc ngẫu nhiên 1 trong 5 task pool và không xem dataset card trước giờ thi.
Metric chính	Metric theo task + time-to-baseline + reproducibility + số lỗi quy trình.
Thời lượng	3-4 giờ (có thể rút còn 2 giờ bằng cách giảm số experiment, không bỏ baseline/error analysis).

Mục tiêu trong ngày: Chứng minh khả năng generalize quy trình chứ không nhớ notebook: nhận dạng task -> metric -> baseline -> model -> validation -> improve -> package.

1. Học lý thuyết gì?

• Không học mới. 15 phút cuối bắt buộc dành cho reproducibility/submission sanity.

2. Triển khai từng bước

1. 0:00-0:20 xác định task type: single text / pair / token labels / multilabel / regression.

2. Chọn metric trước khi model.

3. Dựng baseline phù hợp: TF-IDF/Linear; hoặc simple token/pair baseline nếu task khác.

4. Chọn Transformer head đúng: SequenceClassification / TokenClassification / multilabel / regression.

5. Chạy 2-4 experiments có hypothesis.

6. Error analysis ít nhất 30 samples.

7. Lưu final model/config/predictions và README “how to reproduce”.

8. Tự chấm checklist cuối tháng.

3. Experiments bắt buộc

• Experiment 1: fastest valid baseline.

• Experiment 2: pretrained model.

• Experiment 3: targeted change từ errors.

• Experiment 4: optional speed/ensemble only if time.

4. Tiêu chí PASS

PASS: PASS tổng: baseline hợp lệ <=20-30m tùy task; model pretrained chạy <=2h; >=3 experiments; không leakage; output đúng format; có postmortem và next-step rõ ràng.

5. Nguồn học lý thuyết

• Hugging Face task guides - Chọn đúng task guide khi cần tra API.

• scikit-learn text tutorial - Classical fallback nhanh.

6. Nguồn tham khảo code / API

• HF Datasets loading - Load local/Hub data.

• PyTorch tutorials - Custom code fallback.

7. Lỗi hay mắc

• Cố ép mọi task thành text classification.

• Bỏ baseline vì “Transformer chắc mạnh hơn”.

• Không kiểm output format/row order.

• Dùng hết 4h train model cuối và không kịp submit.

8. Deliverables cuối ngày

• final_mock.ipynb

• final_submission_or_predictions.*

• final_config.json

• 30_day_postmortem.md

• personal_contest_cheatsheet.pdf/md

9. Stretch goal (chỉ làm sau khi PASS)

• Từ postmortem, tạo kế hoạch 2 tuần tiếp theo chỉ tập trung 3 điểm yếu có impact cao nhất.

Checklist Master NLP sau 30 ngày

☐ **1.** Nhận dạng được ít nhất 5 dạng bài: single-text classification, multilabel, token classification/NER, sentence-pair classification, similarity/regression.

☐ **2.** Dựng TF-IDF + LinearSVC/LogReg baseline trong <=20 phút cho single-text classification.

☐ **3.** Tự viết Dataset/DataLoader/training loop PyTorch khi cần custom model.

☐ **4.** Hiểu embedding, RNN/LSTM, attention và Transformer đủ để debug shape/mask/loss.

☐ **5.** Fine-tune Hugging Face sequence classification từ dataset thô mà không cần notebook mẫu.

☐ **6.** Fine-tune PhoBERT/XLM-R cho tiếng Việt và xử lý segmentation/preprocessing đúng model.

☐ **7.** Xử lý multi-label bằng sigmoid/BCEWithLogitsLoss và threshold tuning.

☐ **8.** Xử lý NER bằng token-label alignment và entity-level F1.

☐ **9.** Xử lý sentence pairs/STS với pair tokenizer và metric phù hợp.

☐ **10.** Phát hiện leakage/duplicate/group split và thiết kế validation phù hợp.

☐ **11.** Dùng error analysis để chọn experiment thay vì tuning ngẫu nhiên.

☐ **12.** Biết khi nào K-Fold/ensemble đáng chi phí.

☐ **13.** Tối ưu dynamic padding/mixed precision/gradient accumulation theo GPU.

☐ **14.** Luôn tạo output đúng format, row order, no-NaN và có config tái hiện.

Cheat sheet khi nhận đề NLP trong contest

Bước 1 - Task	1 text? 2 texts? labels per token? nhiều nhãn? output float?
Bước 2 - Metric	Đọc đúng metric và direction (maximize/minimize); viết evaluation function ngay.
Bước 3 - Leakage	ID/group/time/duplicate? preprocessing fit ở đâu?
Bước 4 - Baseline	TF-IDF word+char + linear model nếu text classification; baseline task-specific nếu NER/pair.
Bước 5 - Transformer	Chọn head đúng + model vừa GPU/time; đừng bắt đầu bằng model lớn.
Bước 6 - Errors	Đọc prediction sai/confidence; nhóm lỗi; chọn experiment có lý do.
Bước 7 - Cost	Ước lượng runtime trước khi train; giữ thời gian cho final submit.
Bước 8 - Submit	Row count/order, columns, dtype, NaN, label mapping, file path/name.

Bộ nguồn học nên bookmark

- OlpAI 2026 - Quy chế/Kế hoạch - Bám thể lệ chính thức.
- Stanford CS224N - Lý thuyết deep NLP: embeddings, RNN, attention, Transformer.
- scikit-learn - Working With Text Data - Classical NLP pipeline.
- PyTorch Learn the Basics - Framework fundamentals.

- PyTorch Deep Learning for NLP - Embedding/LSTM NLP code.
- Hugging Face Course - Transformers/tokenizers/fine-tuning.
- HF Text Classification - Sequence classification guide.
- HF Token Classification - NER guide.
- HF GPU Efficiency - Tối ưu memory/speed.
- PhoBERT official - Vietnamese pretrained model.

Dataset index

- IMDb - Sentiment binary.
- SMS Spam UCI - Small binary/noisy text.
- AG News - 4-class topic classification.
- WikiText - Corpus/embedding.
- SST-2 - Sentiment/Transformer basics.
- UIT-VSFC - Vietnamese feedback.
- Jigsaw Toxic - Multi-label.
- CoNLL-2003 - NER/token classification.
- GLUE QQP - Sentence-pair paraphrase.
- GLUE STS-B - Semantic similarity regression.
- Disaster Tweets - Noisy social text contest.

Ghi chú về mốc metric

Các mốc metric trong roadmap (nếu có) là mốc luyện tập tham khảo, không phải ngưỡng chính thức của OlpAI. Dataset split, seed, model version và compute khác nhau sẽ làm điểm thay đổi. Trong contest, metric của đề bài luôn là chuẩn duy nhất.